Synaptiq — Production Architecture

HiDevs × Mastra Hackathon | Round 1 Submission

Team: Synaptiq
Stack: Mastra · Qdrant · Enkrypt AI · Next.js · PostgreSQL · Redis
Problem domain: AI-powered adaptive tutoring for K–12 and competitive exam learners

---

1. Problem & Impact

Indian K–12 and competitive-exam students lose 40–60% of new material within 48 hours because tutoring apps deliver static content, not personalized re-sequencing tied to forgetting curves and mastery gaps. Synaptiq is a TypeScript-native AI tutoring system that replaces one-size-fits-all chatbots with a Mastra-orchestrated multi-agent pipeline: it diagnoses learning style, sequences curriculum by Bloom level and IRT difficulty, retrieves grounded explanations from Qdrant, and gates every turn through Enkrypt AI before a student sees it. Target users are self-paced learners (JEE/NEET/CBSE) who need adaptive remediation without a human tutor. Measurable outcomes are mastery score delta, time-to-remediation, and hallucination block rate.

---

2. Agent Workflows

A session flows through five stages:

1. **Onboarding** — Identity Agent collects grade, goals, and prior scores, writing a learner profile to Postgres and seeding Qdrant with an empty `learner_context` namespace.
2. **Intent routing** — Supervisor Agent classifies the turn (explain, practice, assess, review) and fans out to specialists.
3. **Adaptive planning** — Learning Style Detector, Forgetting-Curve Scheduler, and IRT Calibrator run in parallel to produce a lesson plan (topic order, modality, difficulty θ).
4. **Delivery** — Tutor Agent generates explanations; on visual or kinesthetic style hits, it calls Multimodal Tools (diagram generator, code sandbox) before responding.
5. **Assessment loop** — Assessment Agent emits questions; Bloom Classifier tags cognitive level; wrong answers trigger a branch into Remediation Agent, which queries Qdrant for prerequisite chunks and schedules a spaced-repetition card in Redis.

Agent Justifications (Pain Point → Agent)

Agent	Justification
Supervisor Agent	Students ask ambiguous questions ("help with calculus") — without intent routing, 30%+ of tutor tokens are wasted on the wrong modality.
Learning Style Detector	Static curricula ignore that visual learners retain 65% more when concepts are diagrammed — it switches Tutor Agent to Mermaid/code tools automatically.
Curriculum Sequencer	Exam syllabi are DAGs, not lists — it topologically orders prerequisites so students never hit advanced topics with unmet dependencies.
Tutor Agent	Explanation quality collapses without retrieved context — it grounds every answer in Qdrant chunks instead of parametric memory.

Assessment Agent	Passive reading produces an illusion of competence — it forces active recall, the only signal IRT can calibrate on.
Remediation Agent	A single wrong answer often masks a deeper gap two topics back — it walks the prerequisite graph in Qdrant to find the true failure point.
Progress Tracker	Students abandon apps when progress feels invisible — it surfaces mastery %, streak, and next-review ETA from Postgres + Redis.
Bloom Classifier	"Understanding" at Remember level ≠ Apply level — mis-tagged questions let students pass drills while failing exam-style problems.
IRT Calibrator	Fixed difficulty frustrates both tails — it adjusts item difficulty θ in real time so questions sit in the learner's zone of proximal development.
Forgetting-Curve Scheduler	~60% of retention loss happens in the first 48 hours (Ebbinghaus) — it re-sequences review cards before decay, not on a fixed calendar.
Spaced-Repetition Engine	Cramming beats long-term retention — it writes SM-2 intervals to Redis and surfaces due cards on session start.
RAG Retriever Agent	Textbook PDFs are too large for context windows — it runs hybrid dense+metadata-filtered Qdrant queries scoped to subject and grade.
Identity / Onboarding Agent	Personalization without a profile is guesswork — it bootstraps learner_profiles and seeds Qdrant namespaces per user.

---

3. Mastra Orchestration

Synaptiq registers all agents on a single `Mastra` instance with shared `PostgresStore` (conversation threads) and `QdrantVector` (semantic memory).

Curriculum Workflow — `createWorkflow` graph:

```
onboardingStep
  .then(styleDetectStep)
  .parallel([irtStep, forgettingCurveStep])
  .then(sequencerStep)
  .branch({ practice: assessmentBranch, explain: tutorBranch })
```

Tutor Branch — calls `tutorAgent.generate()` with `memory: { threadId, resourceId }` for session continuity across days.

Multimodal Tools — `createTool` definitions (`generateDiagramTool` , `runCodeSandboxTool`) attached to Tutor Agent; sandbox execution uses `requireToolApproval: true` with `workflow.suspend()` until the student confirms.

Agentic RAG Pipeline — `createVectorQueryTool({ vectorStoreName: 'qdrant', indexName: 'curriculum_chunks' })` on RAG Retriever Agent, with `QDRANT_PROMPT` injected into agent instructions for filter-safe queries.

Supervisor pattern — Supervisor Agent delegates via sub-agent tool calls rather than a monolithic prompt.

Streaming — frontend subscribes to `tutorAgent.stream()` SSE from the Mastra Hono server adapter.

Observability — every workflow step emits to Mastra tracing with `enkrypt_check_id` and `qdrant_hit_ids` as span attributes.

Mastra Primitives Used

Primitive	Where Used
`Mastra()` + `registerAgent`	Root orchestrator instance
`createWorkflow` + `.then()/.branch()/.parallel()`	Curriculum + assessment pipelines
`createStep`	Each workflow stage (onboarding, IRT, tutor, remediate)
`Agent.generate()` / `Agent.stream()`	Tutor, Assessment, Supervisor agents
`createTool`	Diagram generator, code sandbox
`createVectorQueryTool`	RAG Retriever Agent
`Memory` + `PostgresStore`	Cross-session conversation continuity
`QdrantVector` (`@mastra/qdrant`)	Semantic memory backend
`suspend()` / `resume()`	Human-in-the-loop code sandbox approval
Observability spans	Demo debugging + judging metrics

---

4. Qdrant Memory & Retrieval

Four Qdrant collections, all accessed through `@mastra/qdrant` `QdrantVector` registered on the Mastra instance:

Collection	Contents	Retrieval Pattern
`curriculum_chunks`	Textbook/syllabus chunks (1536-dim, text-embedding-3-small)	Filters: subject, grade, bloom_level, topic_id
`learner_misconceptions`	Wrong-answer explanations + error patterns per user	Filter: user_id; upserted after each failed assessment
`session_summaries`	Compressed turn summaries for long sessions	Hybrid: vector similarity + thread_id filter
`multimodal_assets`	Captions/embeddings of generated diagrams	Linked to chunk_id for re-use

Ingestion: PDFs → Mastra `MDocument` chunker → `embed()` → `store.upsert()`

Query path: RAG Retriever Agent calls `createVectorQueryTool` with `topK: 5`, optional reranker; retrieved chunk_ids passed as structured context to Tutor Agent and as `context` string to Enkrypt hallucination check.

Redis (Upstash): caches hot Qdrant results (TTL 15 min) and stores SM-2 review queues — latency layer for due-card lookups, not a replacement for Qdrant.

Write-back: after remediation, misconception vectors are upserted so the same error pattern surfaces faster on the next session.

---

5. Enkrypt AI Evaluation Layer

Enkrypt is a first-class pipeline stage — not a sidecar. Every student turn passes through policy bundle `synaptiq-tutor-v1` (via `POST /guardrails/add-policy`).

Pipeline Stages

```
Student Input
    │
    ▼
┌──────────────────────────────────────────┐
│ STAGE 1 — Pre-Ingress (sync block)       │
│  • POST /guardrails/pii (mutate request) │
│  • POST /guardrails/detect               │
│    (jailbreak + injection + toxicity)    │
└──────────────────────────────────────────┘
    │
    ▼
  Mastra Supervisor → Specialist Agents → Qdrant RAG
    │
    ▼
┌──────────────────────────────────────────┐
│ STAGE 2 — Post-Retrieval (sync)          │
│  • POST /guardrails/adherence            │
│    (response vs. RAG context chunks)     │
└──────────────────────────────────────────┘
    │
    ▼
  Tutor Agent .generate() / .stream()
    │
    ▼
┌──────────────────────────────────────────┐
│ STAGE 3 — Post-Generation (sync)         │
│  • POST /guardrails/hallucination        │
│  • POST /guardrails/relevancy            │
│  • POST /guardrails/detect (toxicity)    │
│  • POST /guardrails/pii (mutate output)  │
└──────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────┐
│ STAGE 4 — Policy Gate (sync)             │
│  • POST /guardrails/policy/detect        │
│    (policy: synaptiq-tutor-v1)           │
└──────────────────────────────────────────┘
    │
    ▼
  Stream to Frontend (or fallback path)
```

Guardrail Policies

Policy	Endpoint	Purpose
PII Redaction	`/guardrails/pii`	Anonymize names, emails, phones before LLM; de-anonymize on display
Jailbreak / Prompt Injection	`/guardrails/detect`	Block adversarial prompts attempting to override tutor instructions
Toxicity / NSFW	`/guardrails/detect`	Block harmful content, especially critical for minor users
Hallucination Detection	`/guardrails/hallucination`	Compare response against RAG context chunks
Context Adherence	`/guardrails/adherence`	Verify response stays within retrieved material
Relevancy	`/guardrails/relevancy`	Ensure answer addresses the student's actual question
Custom Policy Bundle	`/guardrails/policy/detect`	Composite synaptiq-tutor-v1 policy

Failure Actions

Check	Threshold	On Fail
PII (request)	entity detected	**Mutate** — replace with tokens; store enkrypt_key in Redis
Jailbreak / injection	score > 0.85	**Block** — safe refusal; log to safety_events
Toxicity / NSFW	score > 0.8	**Block** — escalate flag for minors; no retry
Adherence (post-RAG)	score < 0.7	**Retry** — widen topK once; then escalate to "insufficient source material"
Hallucination	is_hallucination: 1	**Retry** — strict_grounding prompt; second fail → block + workflow.suspend()
Relevancy	score < 0.6	**Retry** once with Supervisor re-routing
PII (response)	entity detected	**Mutate** before streaming

Feedback loop: Enkrypt results written to `safety_events` (Postgres) and injected as `lastGuardrailResult` into next Supervisor context. Repeated hallucination flags lower Tutor temperature and force vector query on every sentence. Workflow `.branch()` reads `enkrypt_passed: boolean` to choose deliver vs. remediate paths.

---

6. Frontend

Next.js 15 (App Router) on **Vercel**, TypeScript end-to-end.

Pages: `/onboarding` , `/learn/[topicId]` , `/assess/[sessionId]` , `/progress`

Learn view: split pane — chat (Tutor stream via SSE) + dynamic panel (Mermaid diagram / Monaco sandbox from tool-call JSON).

State: TanStack Query for REST/tRPC; `threadId` persisted in localStorage + synced to Postgres.

Auth: Clerk or NextAuth with JWT → `user_id` on every API call.

Safety Badge: green/amber/red indicator driven by Enkrypt stage-4 result, visible during live demo.

Progress dashboard: reads `/api/progress` for IRT θ, Bloom breakdown, and due review cards.

---

7. Backend & APIs

Mastra runs as a **Hono server** on Railway, exposing agents and workflows. Next.js BFF proxies authenticated calls.

Primary REST Endpoints

Method	Path	Purpose
POST	`/api/v1/sessions`	Create session; run Onboarding Workflow
POST	`/api/v1/sessions/:id/message`	Full pipeline: Enkrypt S1 → Mastra → S2–S4 → SSE stream
GET	`/api/v1/sessions/:id/stream`	Resume SSE for in-flight Tutor stream
POST	`/api/v1/assessments/:sessionId/submit`	Submit answer → Assessment Workflow → IRT update
GET	`/api/v1/curriculum/:userId`	Sequenced topic DAG from Curriculum Workflow
GET	`/api/v1/progress/:userId`	Mastery, streak, SM-2 due cards
POST	`/api/v1/reviews/:cardId/complete`	Mark spaced-repetition card done
POST	`/api/v1/ingest`	Admin: PDF → Qdrant upsert
GET	`/api/v1/safety/events/:userId`	Enkrypt audit log
POST	`/api/v1/workflows/:id/resume`	Human-in-the-loop resume after suspend()

Internal: `POST /internal/enkrypt/evaluate` wraps Enkrypt SDK (`GuardrailsClient`) for policy changes without frontend redeploy.

---

8. Database

PostgreSQL (Neon / Supabase) — Relational Source of Truth

```
users
  id, email, grade, exam_target, learning_style, created_at

learner_profiles
  user_id (FK), irt_theta, bloom_distribution (JSONB), style_scores (JSONB)

sessions
  id, user_id, thread_id, status, started_at

messages
  id, session_id, role, content_redacted, enkrypt_key, safety_status, created_at

assessments
```

```
  id, session_id, topic_id, bloom_level, difficulty_theta, correct

mastery_records
  user_id, topic_id, mastery_pct, last_practiced, updated_at

spaced_repetition_cards
  id, user_id, topic_id, interval_days, ease_factor, due_at

safety_events
  id, session_id, stage, check_type, score, action, created_at

curriculum_topics
  id, subject, title, prerequisite_ids[], bloom_level
```

Qdrant — Semantic Layer

Four collections with payload indexes on `user_id`, `subject`, `grade`, `topic_id` (see Section 4).

Redis (Upstash)

- `session:{id}:enkrypt_key` — PII deanonymization keys
- `reviews:due:{userId}` — sorted set by timestamp for SM-2 cards
- Qdrant query cache (TTL 15 min)
- Rate limits (100 req/min/user)

---

9. Deployment

Layer	Host	Rationale
Frontend (Next.js)	Vercel	Edge SSR, instant preview deploys per PR
Mastra API + agents	Railway (Docker)	Long-running Node process; .stream() and workflow state need persistent workers
PostgreSQL	Neon	Branching DB per preview env (dev / staging / prod)
Redis	Upstash	Serverless Redis, same region as Railway
Qdrant	Qdrant Cloud	Managed cluster; separate collections per env
Enkrypt AI	api.enkryptai.com	External SaaS; API key per env in Railway secrets

CI/CD Flow

```
git push → GitHub Actions
  → lint → typecheck → test
  → Docker build → Railway deploy
  → Vercel auto-deploy (frontend)

Branches:
  main        → staging (Railway + Vercel preview)
```

```
tag v*      → production
feature/*   → Vercel preview → staging Mastra API
```

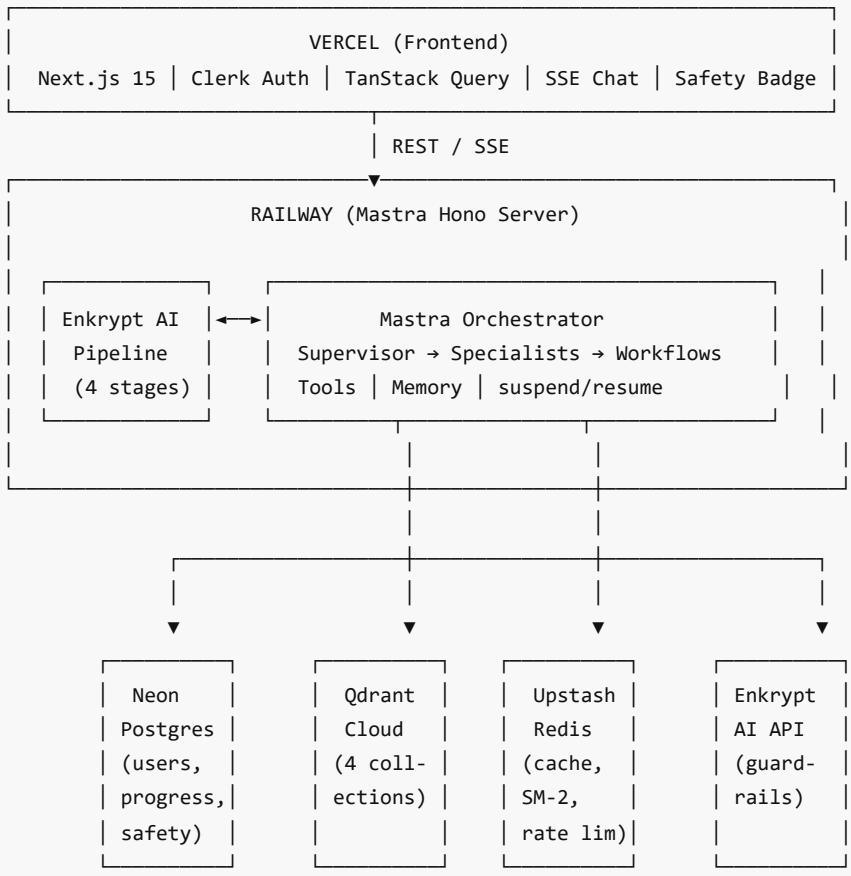
Environment Separation

Environment	Mastra	Qdrant	Postgres	Purpose
development	Local playground	Local / Docker	Local	Agent debugging
staging	Railway (1 replica)	Qdrant Cloud (_staging suffix)	Neon branch	Integration testing
production	Railway (2+ replicas)	Qdrant Cloud	Neon main	Live users

Scaling: Mastra agents scale horizontally on Railway replicas behind a load balancer; workflow state in Postgres via `PostgresStore` ensures suspend/resume works across replicas. Mastra Playground runs locally in dev; production uses Hono adapter only.

---

10. System Architecture Diagram



Judging Alignment

Criterion	Weight	Synaptiq Coverage
Mastra Integration Depth	25%	Workflows, Agents, Tools, Memory, VectorQueryTool, suspend/resume, observability
Qdrant Integration Quality	20%	4 collections, metadata filters, misconception write-back, @mastra/qdrant
Enkrypt AI Coverage	20%	4-stage pipeline, 7 guardrail policies, block/retry/escalate matrix, feedback loop
Agent Output Quality	20%	RAG-grounded Tutor + IRT Assessment + remediation loop
Problem Impact & Novelty	15%	Forgetting-curve scheduling, IRT calibration, 48-hour retention targeting

---

Synaptiq — Built with Mastra, Qdrant, and Enkrypt AI
HiDevs × Mastra Hackathon 2026